DEVOPS OPERATING SYSTEM

OSOMUDEYA ZUDONU

SECTION 2.7: Git & Version Control

Why You Need Git Right Now

You've written scripts, configurations, and documentation. What happens when:

- You break a working script and can't remember what you changed?
- A teammate asks, "Who modified this file and why?"
- You need to try a new approach without losing the current version?
- You want to collaborate on the same Dockerfile?

Without Git: Copy files, rename to `script_v2_final_REAL_final.sh`, lose track, panic.

With Git: Track every change, rollback instantly, collaborate safely.

Git is mandatory for DevOps. Every company uses it. Every interview asks about it.

Part 1: Essential Git Concepts

Concept 1: What is Git?

Git is a time machine for your code.

Think of your project as a timeline:

```
Your Project Timeline
├── Now (latest changes)
├── Yesterday (working version)
├── Last week (before the bug)
└── Last month (original version)
```

Git saves snapshots (commits) so you can travel back in time.

What the system is thinking:

- "User wants to save the current state."
- "Takea snapshot of all files."
- "Store it with a timestamp and message."

- "User can come back to this snapshot anytime"

Real scenario:

```
# Monday: Write Dockerfile, test it, works great
git commit -m "Add working Dockerfile"

# Tuesday: Try to optimize, breaks everything
# Disaster! How do I get back to Monday's version?

# With Git:
git checkout HEAD~1
# You're back to Monday's working version!
```

Understanding HEAD~1:

- `HEAD` = your current commit (where you are now)
- `~1` = go back 1 commit
- `HEAD~1` = the commit before your current one
- So `git checkout HEAD~1` switches you to Monday's version

Why this matters:

- You can experiment freely (always have a way back)
- Never lose working code
- See what changed and when
- Collaborate without fear

Concept 2: Repository (Repo)

A repository is a project folder tracked by Git.

```
my-project/              # Regular folder
├── script.sh
├── config.yaml
└── .git/                 # Git's brain (tracking system)
```

```
    └─ (Git metadata)
```

What the system is thinking:

- "User wants to track this folder."
- "Create .git directory to store tracking info."
- "Now I can remember all changes"
- "Folder is now a repository"

Once you run `git init`, the folder becomes a repository.

Why this matters:

- One command turns any folder into a tracked project
- Git stores all history in the `.git` folder (hidden)
- You can have multiple repositories on your system

Concept 3: Commit

Think of commits as photos of your project over time:

```
Monday:    Commit 1: "Initial Dockerfile"
           ↓ (time passes, you make changes)
Tuesday:   Commit 2: "Add health check"
           ↓ (time passes, you make more changes)
Wednesday: Commit 3: "Fix port configuration"
           ↓ (this is where you are RIGHT NOW)
Current state = Your files as they are today
```

What the system is thinking:

- "User made changes to files."
- "User wants to save this state"
- "Create snapshot with unique ID."
- "Store message explaining what changed."
- "Add timestamp and author."

- "This becomes part of history."

Each commit has:

- **Unique ID (hash):** a3f2b91 (like a fingerprint)
- **Message:** What changed and why
- **Timestamp:** When it happened
- **Author:** Who made the change

Why this matters:

- Every commit is a checkpoint you can return to
- Commit messages explain "why," not just "what."
- History shows who did what and when
- You can see the evolution of your project

Concept 4: The Three Areas

Git has three zones where files live:

```
Working Directory (you edit here)
    ↓ git add
Staging Area (ready to save)
    ↓ git commit
Repository (permanently saved)
```

1. Working Directory - Your files as you edit them (not saved yet)

- This is where you make changes
- Files are "modified" but not tracked

2. Staging Area - Files you've marked as "ready to save" with `git add`

- Files are "staged" and ready to commit
- You can review what will be saved

3. Repository - Files permanently saved with `git commit`

- Files are "committed" and part of history
- This is permanent (until you change it again)

What the system is thinking:

- "User edited files in working directory"
- "User runs git add → move files to staging area"
- "User runs git commit → save staged files to repository"
- "Now files are permanently tracked"

Workflow: Edit files → git add to stage them → git commit to save them permanently.

Why this matters:

- Staging lets you review before committing
- You can stage some files but not others
- Commits are intentional (not automatic)
- You control what gets saved

Concept 5: Branches

Branches let you work on different versions of your project at the same time.

```
main branch (stable, production-ready)
    │
    ├── feature-branch (experimenting with new feature)
    │
    └── bugfix-branch (fixing a bug)
```

What the system is thinking:

- "User wants to try something new"
- "Create new branch from current state"
- "User can switch between branches"
- "Changes on one branch don't affect others."
- "When ready, merge branch back to main"

The main branch holds the stable, production ready code.

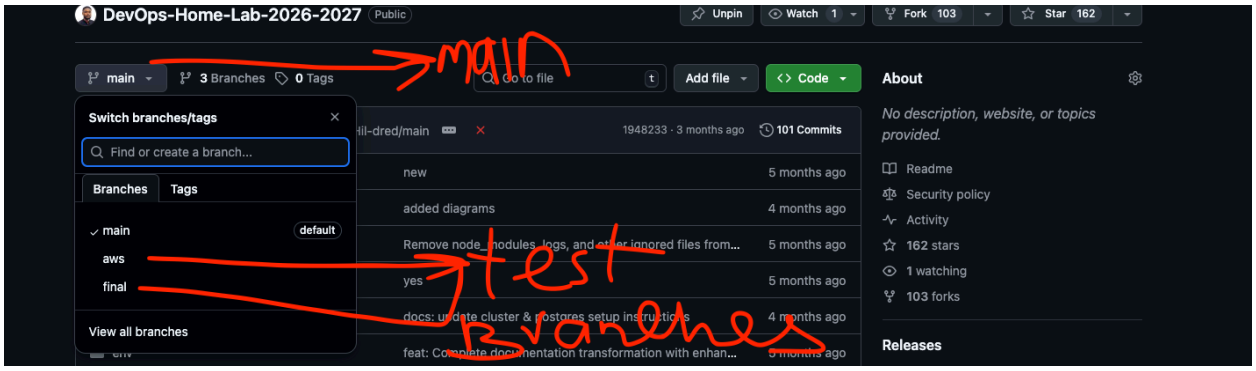
A feature branch is where you test new ideas without affecting the main branch.

Why this matters:

- Experiment without breaking the working code
- Work on multiple features simultaneously
- Keep main branch always deployable
- Collaborate without conflicts

Example:

```
main: Working health check script
  ↓
feature: Try adding email alerts
  ↓
If it works: Merge to main
If it breaks: Delete the branch, main is safe
```



Concept 6: Remote Repository

Your local repository can sync with a remote one, such as GitHub.

```
Local (your laptop/EC2) ←→ Remote (GitHub)
```

What the system is thinking:

- "User wants to backup work to GitHub"

- "git push → upload local commits to remote"
- "git pull → download remote commits to local"
- "Keep local and remote in sync."

git push uploads your local commits to GitHub.

git pull downloads new commits from GitHub.

Why this matters:

- **Backup:** Code safe even if laptop/server dies
- **Collaboration:** Multiple people can work together
- **Portfolio:** Show your work to employers
- **Access:** Work from anywhere

Part 2: Essential Git Commands

Setup Commands (One-Time)

```
# Configure your identity
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Verify configuration
git config --list
```

Expected output:

```
user.name=Your Name
user.email=your.email@example.com
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
```

What this does:

- Sets your name and email for all Git repositories
- Appears in every commit you make
- `--global` means it applies to all repos on this machine

Repository Commands

```
# Create new repository
git init
```

Expected output:

```
Initialized empty Git repository in /home/ubuntu/devops-tasks/.git/
```

What this does:

- Creates `.git` folder (hidden)
- Turns the current directory into a Git repository
- Ready to start tracking files

```
# Clone existing repository
git clone https://github.com/Ship-With-Zee/H-Game.git
```

Expected output:

```
Cloning into 'H-Game'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (82/82), done.
remote: Compressing objects: 100% (58/58), done.
remote: Total 82 (delta 11), reused 82 (delta 11), pack-reused 0
(from 0)
Receiving objects: 100% (82/82), 719.17 KiB | 31.27 MiB/s, done.
Resolving deltas: 100% (11/11), done.
```

What this does:

- Downloads the entire repository from GitHub
- Creates a local copy with all history
- Sets up a remote connection automatically

```
# Check repository status
git status
```

Expected output:

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working
directory)

        modified:   scripts/backup-logs.sh

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        scripts/new-script.sh

no changes added to commit (use "git add" to stage)
```

What each part means:

- `On branch main` = You're on the main branch
- `Changes not staged` = Files modified but not staged
- `Untracked files` = New files Git doesn't know about yet
- `no changes added to commit` = Nothing staged for commit

Basic Workflow Commands

```
# See what changed
```

```
git status                      # Shows which files were modified
git diff                        # Shows the exact changes inside the
files
```

git diff example:

```
git diff scripts/backup-logs.sh
```

Expected output:

```
diff --git a/scripts/backup-logs.sh b/scripts/backup-logs.sh
index abc1234..def5678 100644
--- a/scripts/backup-logs.sh
+++ b/scripts/backup-logs.sh
@@ -10,6 +10,7 @@ readonly AGE_DAYS=7
 # Function: Log with timestamp
 log() {
     echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | sudo tee -a
"$SCRIPT_LOG"
+    echo "DEBUG: Logging to $SCRIPT_LOG"  # Added debug line
 }
```

What this shows:

- Lines starting with - = Removed (red in terminal)
- Lines starting with + = Added (green in terminal)
- @@ -10,6 +10,7 @@ = Line numbers where change occurred

```
# Stage files
git add <filename>              # Add/Stage specific file
git add .                       # Add/Stage all changes
```

What the system is thinking:

- "User wants to stage files."
- "Move files from working directory to staging area."

- "Files are now ready to commit."

```
# Commit changes
git commit -m "Your message"  # Saves a snapshot of your work
```

Expected output:

```
[main a3f2b91] Your message
 1 file changed, 5 insertions(+), 2 deletions(-)
```

What this shows:

- `[main a3f2b91]` = Branch and commit hash
- `1 file changed` = How many files in this commit
- `5 insertions, 2 deletions` = Lines added and removed

```
# View history
git log                       # View all past commits
git log --oneline             # Shorter, easier-to-read commit list
```

git log --oneline example:

```
a3f2b91 docs: add comment about error handling to backup script
b7e4c32 feat: add network connectivity check
c9d3e12 Initial commit: DevOps learning scripts and tasks
```

What each part means:

- `a3f2b91` = Commit hash (unique ID)
- `docs:` = Type of change (documentation)
- Rest = Commit message

Branch Commands

```
# List branches
```

```
git branch
```

Expected output:

```
* main
  feature-health-check
  bugfix-port-config
```

What this shows:

- * = Current branch (you're on main)
- Other branches listed below

```
# Create new branch
git branch feature-name

# Switch to branch
git checkout feature-name

# Create and switch (shortcut)
git checkout -b feature-name
```

What the system is thinking:

- "User wants to create a new branch."
- "Copy current state to new branch."
- "User switches to new branch."
- "Changes now happen on this branch, not main."

```
# Merge new branch into current
git merge feature-name
```

Expected output:

```
Updating a3f2b91..c9d3e12
Fast-forward
```

```
 scripts/health-check-v2.sh | 15 +++++++++++++++
 1 file changed, 15 insertions(+)
```

What this shows:

- `Fast-forward` = No conflicts, clean merge
- Files changed and how many lines

```
# Delete branch
git branch -d feature-name
```

Expected output:

```
Deleted branch feature-name (was c9d3e12).
```

What this means:

- Branch deleted (commits still in history)
- Safe to delete after merging

Remote Commands

```
# View remotes
git remote -v
```

Expected output:

```
origin  git@github.com:username/repo.git (fetch)
origin  git@github.com:username/repo.git (push)
```

What this shows:

- `origin` = Default name for remote repository
- `fetch` = Download from remote

- `push` = Upload to remote

```
# Add remote
git remote add origin https://github.com/username/repo.git

# Push to remote
git push origin main
```

Expected output:

```
Enumerating objects: 45, done.
Counting objects: 100% (45/45), done.
Writing objects: 100% (45/45), 12.34 KiB | 2.47 MiB/s, done.
Total 45 (delta 8), reused 0 (delta 0)
To github.com:username/repo.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

What this shows:

- `45 objects` = Commits and files being uploaded
- `[new branch] main -> main` = Created main branch on remote
- `set up to track` = Local main now tracks remote main

```
# Pull from remote
git pull origin main
```

Expected output:

```
remote: Enumerating objects: 5, done.
Updating a3f2b91..d8e9f12
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

What this shows:

- `Updating a3f2b91..d8e9f12` = Moving from old to new commit
- `Fast-forward` = No conflicts, clean update
- Files changed

Undo Commands

```
# Discard uncommitted changes
git checkout -- filename
```

What this does:

- Reverts the file to the last committed version
- **WARNING:** Loses all uncommitted changes
- Use when you made a mistake and want to start over

```
# Unstage file (keep changes)
git reset HEAD filename
```

What this does:

- Removes file from staging area
- Keeps changes in the working directory
- Use when you staged the wrong file

```
# Undo last commit (keep changes)
git reset --soft HEAD~1
```

What this does:

- Removes the last commit from history
- Keeps all changes staged
- Use when the commit message was wrong

```
# View previous version
git checkout COMMIT_HASH
```

What this does:

- Switches to a specific commit
- Files change to match that commit
- Use to see the old version or test something

Part 3: Hands-On Tasks

TASK G1: Set Up Git and Track Your First Project

Business Context (Beginner-Friendly Version)

Ticket ID: DEVOPS-1850
Priority: High
Assignee: You

Situation (What's Actually Happening)

The Problem: Right now, your DevOps scripts and configs only exist on your EC2 instance. If:

- Server crashes → You lose everything
- You break a working script → Can't remember what changed
- Teammate asks, "Who modified this?" → No way to know
- You want to try something new → Afraid to break working code

What You're Building: A version control system that tracks every change, lets you rollback mistakes, and enables collaboration.

Example:

Monday: Write backup script, works perfectly

Tuesday: Try to optimize, breaks everything

Without Git: Start over from scratch (lose Monday's work)

With Git: `git checkout HEAD~1` → Back to Monday's working version

Impact (Why This Matters)

Before (Without Git):

- Lose work if the server crashes
- Can't remember what changed
- Afraid to experiment (might break things)
- No way to collaborate

After (With Git):

- Every change is tracked and saved
- Can rollback to any previous version
- Experiment freely (always have a way back)
- Collaborate safely with others

Your Mission

- Configure Git with your identity
- Create a Git repository for your DevOps work
- Make your first commit
- View Git history
- Understand the basic workflow

Step-by-Step Guide

Step 1: Configure Git

```
# Set your name (appears in commits)
git config --global user.name "Your Name"

# Set your email
git config --global user.email "your.email@example.com"

# Verify configuration
git config --list
```

Expected output:

```
user.name=Your Name
user.email=your.email@example.com
```

What to look for:

- SUCCESS: Your name and email appear in the list
- FAILED: If empty, check you typed the commands correctly

What the system is thinking:

- "User is setting identity."
- "Store name and email globally."
- "Use this for all future commits."

Step 2: Create a Repository for DevOps Work

```
# Navigate to your DevOps tasks directory
cd ~/devops-tasks

# Initialize Git repository
git init

# Check status
git status
```

Expected output:

```
Initialized empty Git repository in /home/ubuntu/devops-tasks/.git/

On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
```

```
        scripts/
        logs/
        configs/

nothing added to commit, but untracked files present (use "git add" to
track)
```

What this shows:

- `Initialized empty Git repository` = Git is now tracking this folder
- `.git/` folder created (hidden, stores all Git data)
- `Untracked files` = Files Git found but isn't tracking yet

What the system is thinking:

- "User wants to track this folder"
- "Create .git directory"
- "Scan folder for files"
- "Report what's untracked"

Step 3: Create .gitignore File

This is Important: It tells Git what NOT to track, like sensitive files with passwords/secrets you don't want pushed to GitHub.

```
# Create .gitignore
cat > .gitignore << 'EOF'
# Logs
*.log
logs/

# Temporary files
*.tmp
*.swp
*~
```

```
# Environment files (contain secrets!)
.env
*.env

# OS files
.DS_Store
Thumbs.db

# Backup files
*.backup
*.bak
EOF
```

Why .gitignore matters:

- **Never commit secrets** (passwords, API keys)
- **Never commit logs** (they change constantly, huge files)
- **Keep the repository clean** (only track important files)

What the system is thinking:

- "User created .gitignore file."
- "Read patterns from .gitignore."
- "Ignore files matching these patterns."
- "Don't track logs, secrets, or temp files."

Common beginner mistake: Creating .gitignore **AFTER** committing secrets.
Solution: Always create `.gitignore` **FIRST**, before first commit.

Step 4: Stage and Commit Your Work

```
# See what files Git found
git status
```

Expected output:

```
On branch main
```

```
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        .gitignore
        scripts/
        configs/

nothing added to commit but untracked files present
```

What this shows:

- Files listed as "untracked" (Git sees them but isn't tracking yet)
- .gitignore is there (good!)
- Files matching .gitignore patterns are NOT listed (they're ignored)

```
# Add all files (except those in .gitignore)
git add .
```

What this does:

- Moves files from "untracked" to "staged."
- Files are now ready to commit
- .gitignore patterns are respected (ignored files not added)

Quick reference:

- `git add -A` = Stage everything (new, modified, deleted)
- `git add .` = Stage everything in current directory only
- `git add <filename>` = Stage one file

```
# Check what's staged
git status
```

Expected output:

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)

        new file:   .gitignore
        new file:   scripts/backup-logs.sh
        new file:   scripts/health-check-v2.sh
        new file:   configs/nginx.conf
```

What to look for:

- **SUCCESS:** Files listed under "Changes to be committed" (staged and ready)
- Files in .gitignore are NOT listed (correctly ignored)

```
# Make your first commit
git commit -m "Initial commit: DevOps learning scripts and tasks."
```

Expected output:

```
[main (root-commit) a3f2b91] Initial commit: DevOps learning scripts and
tasks
 4 files changed, 245 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 scripts/backup-logs.sh
 create mode 100644 scripts/health-check-v2.sh
 create mode 100644 configs/nginx.conf
```

What this shows:

- [main (root-commit) a3f2b91] = Branch, first commit, commit hash
- 4 files changed, 245 insertions = What was saved
- `create mode` = New files created

What the system is thinking:

- "User wants to save the current state."
- "Take a snapshot of all staged files."
- "Createa commit with a message and timestamp."
- "Store in repository history"

```
# Verify commit was created
git log
```

Expected output:

```
commit a3f2b91abc123def456789...
Author: Your Name <your.email@example.com>
Date:   Mon Dec 4 10:00:00 2025 UTC

    Initial commit: DevOps learning scripts and tasks
```

What this shows:

- Commit hash (unique ID)
- Author (you)
- Date and time
- Commit message

Step 5: Make a Change and Commit Again

```
# Modify a script (example)
echo "# Updated with better error handling" >> scripts/backup-logs.sh

# See what changed
git status
```

Expected output:

```
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
```

```
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   scripts/backup-logs.sh

no changes added to commit (use "git add" to stage)
```

What this shows:

- `modified: scripts/backup-logs.sh` = File was changed
- `Changes not staged` = Changes exist but not staged yet

```
# See the exact changes
git diff scripts/backup-logs.sh
```

Expected output:

```
diff --git a/scripts/backup-logs.sh b/scripts/backup-logs.sh
index abc1234..def5678 100644
--- a/scripts/backup-logs.sh
+++ b/scripts/backup-logs.sh
@@ -45,6 +45,7 @@ rotate_logs() {
     log "Searching for .gz files older than $AGE_DAYS days..."
+    echo "# Updated with better error handling"
```

What this shows:

- Line starting with `+` = Added (the new line you added)
- Shows exactly what changed

```
# Stage the change
git add scripts/backup-logs.sh

# Commit with descriptive message
git commit -m "docs: add comment about error handling to backup
script"

# View history
```

```
git log --oneline
```

Expected output:

```
b7e4c32 docs: add comment about error handling to backup script
a3f2b91 Initial commit: DevOps learning scripts and tasks
```

What this shows:

- Two commits now (newest first)
- Each has a unique hash and message
- You can see the progression

Common Beginner Mistakes

1. **Forgetting to stage files** → `git commit` says "nothing to commit."
    - **Solution:** Always `git add` before `git commit`
2. **Committing secrets** → Exposed passwords on GitHub
    - **Solution:** Always create `.gitignore` FIRST, before first commit
3. **Bad commit messages** → "fix" or "update" tells you nothing
    - **Solution:** Use descriptive messages: "fix: correct port binding in Dockerfile."
4. **Not checking status** → Don't know what changed
    - **Solution:** Always `git status` before committing

Acceptance Criteria

Check all boxes:

- ☐ Git configured with your name and email
- ☐ Repository initialized in ~/devops-tasks
- ☐ .gitignore created (protects secrets)
- ☐ Initial commit made with all your work

- ☐ Second commit made after modifying a file
- ☐ Can view Git history
- ☐ Understand what git add, commit, and status do

What You Learned

Technical Skills:

- Configuring Git
- Initializing repositories
- Using .gitignore for security
- Staging and committing changes
- Viewing Git history
- Reading diffs

Critical Concepts:

- Git tracks changes, not just files
- Commits are snapshots you can return to
- .gitignore prevents committing secrets
- Good commit messages explain "why."

Real World Application:

- This is exactly how professionals use Git
- Every change is tracked with context
- You can always rollback mistakes
- Audit trail of who did what when

TASK G2: Work with Branches

Business Context (Beginner-Friendly Version)

Ticket ID: DEVOPS-1892
Priority: Medium
Assignee: You

Situation

The Problem: You want to experiment with improving the health check script, but you're afraid of breaking the working version. If you modify it directly and it breaks, you lose the working code.

What You're Building: A safe way to experiment using branches, work on improvements without affecting the stable version.

Example:

```
main branch: Working health check script (don't touch!)
  ↓
feature branch: Try adding email alerts
  ↓
If it works: Merge to main
If it breaks: Delete branch, main is still safe
```

Impact

Before (Without Branches):

- Afraid to experiment (might break working code)
- Have to copy files manually
- Lose track of what changed
- Can't work on multiple features

After (With Branches):

- Experiment freely (main branch stays safe)
- Work on multiple features simultaneously

- Easy to compare versions
- Merge when ready, delete if not

Your Mission

- Create a feature branch
- Make improvements on the branch
- Test the changes
- Merge back to main
- Delete the feature branch

Step-by-Step Guide

Step 1: Check Current Branch

```
# See what branch you're on
git branch
```

Expected output:

```
* main
```

What this shows:

- * = Current branch (you're on main)
- Only one branch exists so far

Step 2: Create Feature Branch

```
# Create and switch to new branch
git checkout -b improve-health-check

# Verify you switched
git branch
```

Expected output:

```
* improve-health-check
  main
```

What this shows:

- * = You're now on improve-health-check branch
- main = Still exists, but you're not on it

What the system is thinking:

- "User wants new branch"
- "Copy current state to new branch"
- "Switch to new branch"
- "Changes now happen on this branch, not main"

Now you're on the new branch. Changes won't affect the main.

Step 3: Make Changes

```
# Edit the health check script
vim ~/devops-tasks/scripts/health-check-v2.sh
```

Add a new check (example):

```
# Add this function before main()
check_network() {
    log "INFO" "=== Checking Network Connectivity ==="

    if ping -c 1 8.8.8.8 &> /dev/null; then
        log "INFO" "Network connectivity: OK"
    else
        log "ERROR" "No internet connectivity"
        EXIT_CODE=1
    fi
```

```
}
# Add this call in main() function
check_network
```

Save the file, then:

```
# See what changed
git status
git diff scripts/health-check-v2.sh

# Stage the change
git add scripts/health-check-v2.sh

# Commit on feature branch
git commit -m "feat: add network connectivity check to health
monitoring"
```

Expected output:

```
[improve-health-check c9d3e12] feat: add network connectivity check
to health monitoring
 1 file changed, 15 insertions(+)
```

What this shows:

- Commit is on `improve-health-check` branch
- Main branch is unchanged

Step 4: Test Your Changes

```
# Test the improved script
~/devops-tasks/scripts/health-check-v2.sh

# Verify network check runs
```

If it works, continue. If it breaks, you can easily abandon this branch.

What the system is thinking:

- "User is testing changes on feature branch"
- "Main branch is untouched"
- "If the test fails, the user can delete the branch, and the `main` is safe."

Step 5: Switch Back to Main

```
# Switch to main branch
git checkout main

# Check the script
cat ~/devops-tasks/scripts/health-check-v2.sh | grep "check_network"
```

Expected output:

```
(empty - no output)
```

What this means:

- No `check_network` function found
- It only exists on the feature branch
- Your main branch is unchanged and safe

What the system is thinking:

- "User switches to main branch."
- "Files change to match main branch state."
- "Feature branch changes are not visible here."

Step 6: Merge Feature Branch

```
# Switch back to main (if not already there)
git checkout main
```

```
# Merge the feature branch
git merge improve-health-check
```

Expected output:

```
Updating b7e4c32..c9d3e12
Fast-forward
 scripts/health-check-v2.sh | 15 +++++++++++++++
 1 file changed, 15 insertions(+)
```

What this shows:

- `Fast-forward` = No conflicts, clean merge
- `1 file changed, 15 insertions` = Changes added to main

What the system is thinking:

- "User wants to merge the feature branch into main."
- "Apply all commits from the feature branch to main."
- "Feature is now part of the main branch."

```
# Check the log
git log --oneline
```

Expected output:

```
c9d3e12 feat: add network connectivity check to health monitoring
b7e4c32 docs: add comment about error handling to backup script
a3f2b91 Initial commit: DevOps learning scripts and tasks
```

What this shows:

- Feature commit is now in main
- History shows the progression

Step 7: Clean Up

```
# Delete the feature branch (no longer needed)
git branch -d improve-health-check

# Verify it's gone
git branch
```

Expected output:

```
* main
```

What this shows:

- Feature branch deleted
- Only the main branch remains
- Commits are still in history (just the branch is gone)

What the system is thinking:

- "User deleted feature branch."
- "Commits are still in the main branch."
- "Branch pointer removed, commits preserved."

Acceptance Criteria

Check all boxes:

- Created feature branch
- Made changes on the feature branch
- The main branch remained unchanged while working
- Successfully merged the feature branch into the main
- Deleted feature branch after merge
- Understand why branches are useful
- Can switch between branches

What You Learned

Technical Skills:

- Creating branches
- Switching between branches
- Merging branches
- Deleting branches
- Branch-based workflow

Branching Strategy:

- `Main` branch stays stable
- `Feature` branches for experiments
- Merge when ready
- Delete after merging

Real World Pattern:

```
main        → Always deployable
  ↓
feature     → Work on new feature
  ↓
test        → Verify it works
  ↓
merge       → Add to main
  ↓
deploy      → Ship to production
```

TASK G3: Connect to GitHub and Push Your Work

Business Context

Ticket ID: DEVOPS-1935
Priority: High
Assignee: You

Situation

The Problem: Your work only exists on your EC2 instance. If:

- Server crashes → You lose everything
- Laptop breaks → Can't access your work
- Need to work from different machine → Files aren't there
- Want to show employers → No way to share your work

What You're Building: A backup and collaboration system using GitHub - your work is safe in the cloud and accessible from anywhere.

Example:

```
Local (EC2): Your scripts and configs
  ↓ git push
GitHub (Cloud): Backup of all your work
  ↓
Benefits: Safe backup, portfolio, collaboration, access from anywhere
```

Impact (Why This Matters)

Before (Without GitHub):

- Work only on one machine
- Lose everything if machine breaks
- Can't collaborate
- No way to show your work

After (With GitHub):

- Work backed up in cloud
- Access from any machine
- Easy collaboration
- Portfolio for employers

Your Mission

- Create GitHub account (if needed)
- Generate SSH key for authentication
- Create GitHub repository
- Push your local work to GitHub
- Verify backup succeeded

Step-by-Step Guide

Step 1: Create a GitHub Account

If you don't have one:

1. Go to https://github.com
2. Click "Sign up" (if you havn't)
3. Follow the registration process
4. Verify your email

If you already have one, skip to Step 2.

Step 2: Generate SSH Key

Why SSH keys:

- More secure than passwords
- No need to type password every time
- Industry standard

```
# Generate SSH key
ssh-keygen -t ed25519 -C "your.email@example.com"

# Press Enter to accept the default location
# Press Enter twice (no passphrase for learning)
```

Expected output:

```
Your identification has been saved in /home/ubuntu/.ssh/id_ed25519
Your public key has been saved in /home/ubuntu/.ssh/id_ed25519.pub
```

What this does:

- Creates two files: private key (keep secret) and public key (share with GitHub)
- Private key stays on your machine
- Public key goes to GitHub

Copy your public key:

```
cat ~/.ssh/id_ed25519.pub
```

Expected output (copy this entire line):

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... your.email@example.com
```

What to look for:

- SUCCESS: Long string starting with `ssh-ed25519`

- This is your public key (safe to share)

Step 3: Add SSH Key to GitHub

1. **In GitHub:**
    - Click your profile picture (top right)
    - Click "Settings"
    - Click "SSH and GPG keys" (left sidebar)
    - Click "New SSH key."
    - Title: "DevOps Lab EC2"
    - Key: Paste the public key you copied
    - Click "Add SSH key"
2. **Test the connection:**

```
ssh -T git@github.com
```

Expected output:

Hi username! You've successfully authenticated, but GitHub does not provide shell access.

What this means:

- SUCCESS: SSH key is working
- You can now push/pull without passwords

If it fails:

- Check key was copied correctly (no extra spaces)
- Verify key is in GitHub → Settings → SSH keys
- Make sure you used the PUBLIC key (id_ed25519.pub), not private key

Step 4: Create GitHub Repository

1. **In GitHub:**
    - Click "+" (top right)
    - Click "New repository"

- Repository name: `devops-learning`
- Description: "DevOps tasks and scripts from The DevOps Career Playbook."
- Public or Private (your choice)
- **Do NOT initialize with README** (you already have content)
- Click "Create repositor.y"

GitHub will show setup commands, but you'll use different ones.

Step 5: Connect Local Repo to GitHub

```
# Add GitHub as remote (replace YOUR_USERNAME with your GitHub
username)
git remote add origin
git@github.com:YOUR_USERNAME/devops-learning.git

# Verify remote was added
git remote -v
```

Expected output:

```
origin  git@github.com:YOUR_USERNAME/devops-learning.git (fetch)
origin  git@github.com:YOUR_USERNAME/devops-learning.git (push)
```

What this shows:

- origin = Name for your remote repository
- fetch = Download from GitHub
- push = Upload to GitHub

Step 6: Push Your Work

```
# Push main branch to GitHub
git push -u origin main
```

Expected output:

```
Enumerating objects: 45, done.
Counting objects: 100% (45/45), done.
Writing objects: 100% (45/45), 12.34 KiB | 2.47 MiB/s, done.
Total 45 (delta 8), reused 0 (delta 0)
To github.com:YOUR_USERNAME/devops-learning.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

What this shows:

- `45 objects` = Your commits and files
- `[new branch] main -> main` = Created main branch on GitHub
- `set up to track` = Local main now tracks remote main

What the system is thinking:

- "User wants to upload local commits to GitHub."
- "Connect to GitHub via SSH"
- "Upload all commits and files"
- "Create main branch on remote"
- "Link local and remote branches"

Step 7: Verify on GitHub

1. **In your browser:**
    - Go to `https://github.com/YOUR_USERNAME/devops-learning`
    - You should see all your files and folders
    - Click on a file to view it
    - Check commit history

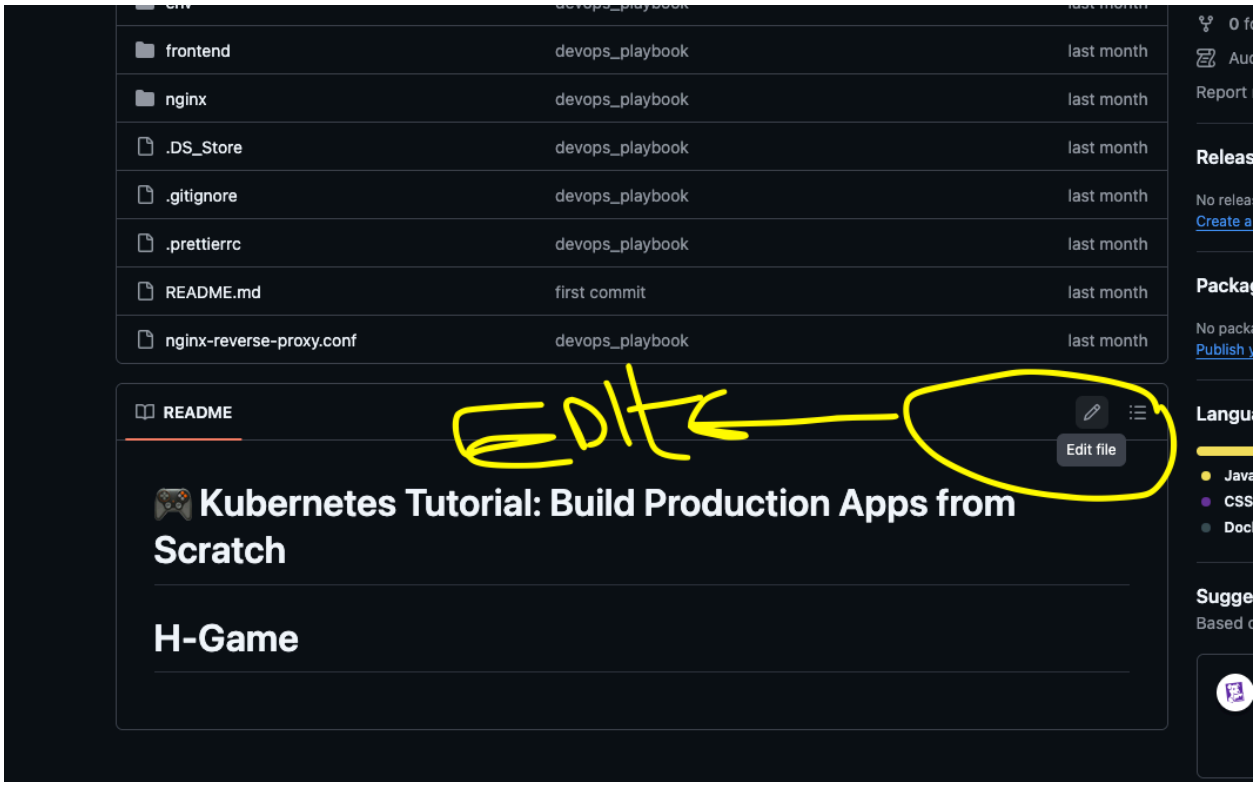
What to look for:

- SUCCESS: All your files are visible
- SUCCESS: Commit history shows your commits
- Your work is now backed up on GitHub

Step 8: Test Pull (Simulating Collaboration)

Make a change on GitHub:

1. Click on any file (e.g., README.md)
2. Click the pencil icon (edit)



3. **Add a line:** `Updated from GitHub web interface`
4. Click "Commit changes."
5. Add a commit message at the commit message pop up

Pull the change to your local machine:

```
# Pull latest changes from GitHub
git pull origin main
```

Expected output:

```
remote: Enumerating objects: 5, done.
```

```
Updating a3f2b91..d8e9f12
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
```

What this shows:

- `Updating a3f2b91..d8e9f12` = Moving to new commit
- `Fast-forward` = No conflicts, clean update

Verify the change locally:

```
cat README.md
```

You should see the line you added on GitHub.

Common Beginner Mistakes

1. **Wrong remote URL** → Push fails
    - **Solution:** Use `git@github.com:USERNAME/REPO.git` (SSH) or `https://github.com/USERNAME/REPO.git` (HTTPS)
2. **SSH key not added to GitHub** → Authentication fails
    - **Solution:** Always add public key to GitHub → Settings → SSH keys
3. **Not pulling before push** → "Updates were rejected."
    - **Solution:** `git pull` first, then `git push`
4. **Pushing to the wrong branch** → Changes go to the wrong place
    - **Solution:** Always check `git branch` before pushing

Acceptance Criteria

Check all boxes:

- ☐ GitHub account created
- ☐ SSH key generated
- ☐ SSH key added to GitHub
- ☐ Can authenticate with GitHub (ssh -T works)
- ☐ Created a remote repository
- ☐ Connected local repo to GitHub
- ☐ Pushed all commits to GitHub
- ☐ Can view files on GitHub
- ☐ Successfully pulled changes from GitHub
- ☐ Understand push vs pull

What You Learned

Technical Skills:

- Generating SSH keys
- Configuring GitHub authentication
- Adding remotes
- Pushing to remote
- Pulling from remote
- Syncing local and remote

GitHub Workflow:

```
Local Changes → Commit → Push → GitHub
                                   ↓
              Pull ← Remote Changes
```

Real World Benefits:

- **Backup:** Code safe even if laptop/server dies
- **Portfolio:** Show your work to employers
- **Collaboration:** Multiple people can work together
- **Access:** Work from anywhere

TASK G4: Clone the H-Game Repository

Business Context (Beginner-Friendly Version)

Ticket ID: DEVOPS-1978
Priority: High
Assignee: You

Situation

The Problem: The Humor Memory Game application is ready for containerization (Section 3). You need the codebase to work with, but it's on GitHub, not your local machine.

What You're Doing: Cloning the repository so you have a local copy to work with for the Docker section.

Example:

```
GitHub: H-Game repository (original code)
  ↓ git clone
Your EC2: Local copy to modify and containerize
  ↓
You'll: Build Docker images, create docker-compose.yml, deploy
```

Impact and Why This Matters

Before (Without Clone):

- Can't work on the project locally
- No way to modify and test
- Can't prepare for Docker section

After (With Clone):

- Have the full codebase locally
- Can explore and understand structure

- Ready to containerize in Section 3

Your Mission

- Fork the H-Game repository (get your own copy)
- Clone your fork to the local machine
- Explore the codebase structure
- View Git history
- Prepare for the Docker section

Step-by-Step Guide

Step 1: Fork the Repository on GitHub

1. In your browser:
    - Go to: https://github.com/Ship-With-Zee/H-Game
    - Click the "Fork" button (top right)
    - Select your account as the destination
    - Wait for GitHub to create your fork

You now have: https://github.com/YOUR_USERNAME/H-Game

Why fork:

- You get your own copy to modify
- You can experiment without affecting the original
- You can submit improvements back (pull requests)

Step 2: Create Projects Directory

```
# Create directory for cloned projects
mkdir -p ~/projects
cd ~/projects
```

Step 3: Clone Your Fork

```
# Clone YOUR fork (replace YOUR_USERNAME)
git clone git@github.com:YOUR_USERNAME/H-Game.git

# Navigate into repository
cd H-Game
```

Expected output:

```
Cloning into 'H-Game'...
remote: Enumerating objects: 82, done.
remote: Counting objects: 100% (82/82), done.
remote: Compressing objects: 100% (58/58), done.
remote: Total 82 (delta 11), reused 82 (delta 11), pack-reused 0 (from 0)
Receiving objects: 100% (82/82), 719.17 KiB | 31.27 MiB/s, done.
Resolving deltas: 100% (11/11), done.
```

What this shows:

- Downloaded the entire repository
- All files and history are now local

Step 4: Verify Clone Success

```
# Check remote configuration
git remote -v
```

Expected output:

```
origin  git@github.com:YOUR_USERNAME/H-Game.git (fetch)
origin  git@github.com:YOUR_USERNAME/H-Game.git (push)
```

What this shows:

- `origin` = Your fork (you can push here)

```
# Check current branch
git branch
```

Expected output:

```
* main

# View Git history
git log --oneline
```

This shows the commit history of the project.

Step 5: Add Upstream Remote

This lets you pull updates from the original repo:

```
# Add original repo as 'upstream'
git remote add upstream git@github.com:Ship-With-Zee/H-Game.git

# Verify remotes
git remote -v
```

Expected output:

```
origin    git@github.com:YOUR_USERNAME/H-Game.git (fetch)
origin    git@github.com:YOUR_USERNAME/H-Game.git (push)
upstream  git@github.com:Ship-With-Zee/H-Game.git (fetch)
upstream  git@github.com:Ship-With-Zee/H-Game.git (push)
```

What this means:

- `origin` = Your fork (you can push here)
- `upstream` = Original repo (you can pull updates)

Step 6: Explore the Repository

```
# View project structure
ls -la
```

Expected structure:

```
backend/
frontend/
database/
nginx/
env/
assets/
.gitignore
README.md
```

```
# Check what's ignored
cat .gitignore
```

Notice: Already has proper .gitignore (node_modules, .env, logs)

```
# Explore backend
ls -la backend/
```

Key files:

- `Dockerfile` - Container definition
- `server.js` - API entry point
- `package.json` - Dependencies

```
# View backend Dockerfile
cat backend/Dockerfile
```

Read through it, and you'll understand every line by the end of Section 3.

Acceptance Criteria

Check all boxes:

- ☐ Forked H-Game repository on GitHub
- ☐ Cloned YOUR fork to ~/projects/H-Game
- ☐ Verified remote configuration (origin + upstream)
- ☐ Explored project structure
- ☐ Read through Dockerfiles
- ☐ Understand repository workflow
- ☐ Ready for Docker section

What You Learned

Git Workflow:

- Forking repositories
- Cloning your fork
- Setting up upstream remote
- Understanding project structure

Collaboration Pattern:

```
Fork → Clone → Work → Push → (Optional) Pull Request
                ↓
        Pull updates from upstream
```

Real World Application:

- This is how open source works
- This is how teams collaborate
- Fork = your personal copy
- Upstream = authoritative source

TASK G5: Handle Common Git Scenarios

Business Context

Ticket ID: DEVOPS-2015
Priority: Medium
Assignee: You

Situation

The Problem: You'll make mistakes with Git. Everyone does. You need to know how to fix them quickly.

What You're Learning: Common scenarios and how to handle them safely.

Your Mission

- Learn to undo uncommitted changes
- Fix commit mistakes
- View old versions
- Protect secrets with .gitignore
- Move commits between branches

Common Scenarios

Scenario 1: Undo Uncommitted Changes

Problem: You modified a file but want to discard the changes.

```
# Make a change you'll want to undo
echo "This is a mistake" >> ~/devops-tasks/scripts/backup-logs.sh

# See the change
git status
git diff scripts/backup-logs.sh

# Undo the change
git checkout -- scripts/backup-logs.sh
```

```
# Verify it's gone
git status
```

What this does:

- Reverts file to the last committed version
- **WARNING:** Loses all uncommitted changes
- Use when you made a mistake and want to start over

Scenario 2: Amend Last Commit

Problem: You forgot to add a file to your last commit.

```
cd ~/devops-tasks
```

```
# Make a commit
echo "# Network troubleshooting notes" > network-notes.md
git add network-notes.md
git commit -m "docs: add network notes"

# Oops! Forgot to add something
echo "- Always check Security Groups first" >> network-notes.md

# Add to the SAME commit (amend)
git add network-notes.md
git commit --amend --no-edit

# View history (still one commit)
git log --oneline
```

What this does:

- Updates the last commit instead of creating a new one
- `--no-edit` keeps the same commit message
- Use when the commit message was wrong or you forgot a file

Scenario 3: View Old Version

Problem: I want to see what a file looked like before the changes.

```
# View commit history
git log --oneline

# View file from 2 commits ago
git show HEAD~2:scripts/health-check-v2.sh

# View entire project state from 2 commits ago
git checkout HEAD~2

# Look around
ls scripts/

# Return to latest version
git checkout main
```

What this does:

- `git show COMMIT:FILE` = View specific file from specific commit
- `git checkout COMMIT` = Switch to that commit (temporary)
- `git checkout main` = Return to the latest version

Scenario 4: Create .gitignore for Secrets

Problem: Almost committed sensitive files.

```
cd ~/projects/H-Game
```

```
# Create fake secret file
echo "DATABASE_PASSWORD=super_secret_123" > .env

# Check status (BAD - shows .env)
git status
```

Create .gitignore BEFORE committing:

```
cat > .gitignore << 'EOF'
# Environment variables (NEVER commit these!)
.env
.env.*
*.env

# Logs
*.log
logs/

# Dependencies
node_modules/
package-lock.json

# OS files
.DS_Store
Thumbs.db
EOF

# Add .gitignore
git add .gitignore
git commit -m "security: add .gitignore to prevent committing
secrets"

# Verify .env is ignored
git status
# Should NOT show .env
```

What this does:

- Protects secrets from being committed
- Always create .gitignore FIRST

Scenario 5: Fix "I Committed to Wrong Branch"

Problem: Made commits on main that should be on a feature branch.

```
cd ~/devops-tasks

# Accidentally on main
git branch
# * main

# Make a change
echo "# TODO: Add email alerts" >> scripts/health-check-v2.sh
git add scripts/health-check-v2.sh
git commit -m "feat: add email alert capability"

# Oops! Should have been on feature branch

# Create branch with current work
git branch add-email-alerts

# Reset main to before the commit
git reset --hard HEAD~1

# Switch to feature branch (has the commit)
git checkout add-email-alerts
git log --oneline
# The commit is here!

# Switch back to main (commit is gone)
git checkout main
git log --oneline
# The commit is NOT here
```

What this does:

- Moves commit from main to feature branch
- `git branch NAME` = Create branch with current commits

- `git reset --hard HEAD~1` = Remove last commit from current branch
- Use when you committed to wrong branch

Acceptance Criteria

Check all boxes:

- ☐ Can discard uncommitted changes
- ☐ Can amend the last commit
- ☐ Can view previous versions
- ☐ Created .gitignore for secrets
- ☐ Understand why .gitignore is critical
- ☐ Can move commits between branches
- ☐ Comfortable with Git commands

What You Learned

Git Operations:

- Discarding changes: `git checkout -- file`
- Amending commits: `git commit --amend`
- Viewing history: `git show`, `git log`
- Time travel: `git checkout COMMIT`
- Branch manipulation: `git reset`, `git branch`

Critical Security:

- NEVER commit .env files
- NEVER commit passwords
- NEVER commit API keys
- Use .gitignore BEFORE first commit

Real World Wisdom:

- Everyone makes mistakes
- Git lets you fix them
- Learn the undo commands

- .gitignore is mandatory

Part 4: Git Best Practices

Writing Good Commit Messages

Bad commit messages:

```
git commit -m "fix"
git commit -m "update"
git commit -m "changes"
```

These tell you nothing!

Good commit messages:

```
git commit -m "fix: correct port binding in backend Dockerfile"
git commit -m "feat: add health check endpoint to API"
git commit -m "docs: update README with setup instructions"
git commit -m "refactor: simplify error handling in backup script"
```

Commit message format:

```
type: brief description (max 50 chars)

Optional longer explanation of what and why.
```

Common types:

- `feat:` New feature
- `fix:` Bug fix
- `docs:` Documentation
- `refactor:` Code cleanup
- `test:` Add tests
- `chore:` Maintenance

What NOT to Commit

Create this .gitignore for every project:

```
# Secrets and credentials
.env
.env.*
*.env
*.pem
*.key
id_rsa*
credentials.json
secrets.yaml

# Logs
*.log
logs/
*.log.*

# Dependencies
node_modules/
venv/
__pycache__/
*.pyc

# OS files
.DS_Store
Thumbs.db
desktop.ini

# IDE
.vscode/
.idea/
*.swp
*.swo
*~

# Build artifacts
dist/
```

```
build/
*.o
*.class

# Temporary files
*.tmp
temp/
*.cache
```

Branching Strategy

For learning/personal projects:

```
main - Always working code
  ↓
feature branches - Experiments and new features
```

For team projects (you'll learn this later):

```
main/master - Production code
  ↓
develop - Integration branch
  ↓
feature/xxx - Individual features
  ↓
hotfix/xxx - Emergency fixes
```

Quick Reference Card

Setup

```
git config --global user.name "Name"
git config --global user.email "email"
```

Create

```
git init                          # New repo
git clone URL                     # Copy remote repo
```

Basic Workflow

```
git status                        # What changed?
git add FILE                      # Stage file
git add .                         # Stage all
git commit -m "message"           # Save snapshot
git log                           # View history
git log --oneline                # Compact history
```

Branches

```
git branch                        # List branches
git branch NAME                   # Create branch
git checkout NAME                 # Switch branch
git checkout -b NAME              # Create + switch
git merge NAME                    # Merge branch
git branch -d NAME                # Delete branch
```

Remotes

```
git remote add origin URL         # Add remote
git push origin main              # Upload
git pull origin main              # Download
git clone URL                     # Copy repo
```

Undo

```
git checkout -- FILE              # Discard changes
git reset HEAD FILE               # Unstage
git commit --amend                # Fix last commit
```

```
git reset --soft HEAD~1          # Undo commit (keep changes)
git reset --hard HEAD~1          # Undo commit (lose changes)
```

Inspect

```
git diff                         # See changes
git show COMMIT                  # View commit
git log --oneline --graph        # Visual history
```

Critical

```
NEVER commit:
- .env files
- Passwords
- API keys
- SSH keys

Always create .gitignore FIRST!
```

Interview Talking Points

"Tell me about your experience with Git. "I use Git daily for all my DevOps work. I've set up repositories, worked with branches for feature development, and pushed my work to GitHub for backup and portfolio purposes.
I'm comfortable with the full workflow from staging changes to merging branches. I also understand security practices like never committing secrets and using .gitignore files."

"Have you used Git in a team environment?" "While most of my learning has been individual, I understand the collaborative workflows, creating feature branches, making commits with clear messages, and merging changes. I've also practiced cloning repositories and pulling changes, which are essential for team collaboration."

Self Check

Before proceeding, ensure you can:

- ☐ Initialize a Git repository

- ☐ Stage and commit changes
- ☐ Write meaningful commit messages
- ☐ Create and merge branches
- ☐ Push to and pull from GitHub
- ☐ Clone repositories
- ☐ Use .gitignore to protect secrets
- ☐ Undo mistakes safely
- ☐ View Git history
- ☐ Understand why version control matters

Take a break. You've built a complete DevOps foundation.

The next phase is where it gets really exciting: containers, orchestration, and production deployments!

Next, Lets learn about how to use the cursor as an IDE for the next task instead of using terminal `ls` and `cd`, etc

SECTION 2.8: Using Cursor IDE

Why Use Cursor?

You've been using the terminal for everything, but an IDE like VS Code and Cursor IDE makes coding faster:

- Visual file navigation (no more `ls` and `cd`)
- Syntax highlighting (easier to read code)
- Built-in terminal (use both in one window)
- Git panel (visual commits instead of commands)
- Search across files (faster than `grep`)

You can still use the terminal; Cursor has one built in. Use what works best.

Setup: Get Your Work into Cursor

Before using Cursor, you need your project on your local machine.

Option 1: Clone from GitHub (If You Pushed in TASK G3)

If you already pushed `~/devops-tasks` to GitHub:

```
# On your local machine (not EC2)
cd ~

git clone git@github.com:YOUR_USERNAME/devops-learning.git
devops-tasks
cd devops-tasks

Then in Cursor: File → Open Folder → ~/devops-tasks
```

Option 2: Start Fresh Locally

If you want to start a new project on your local machine:

```
# On your local machine
mkdir -p ~/devops-tasks
```

```
cd ~/devops-tasks
git init

Then in Cursor: File → Open Folder → ~/devops-tasks
```

Option 3: Work on EC2 (Remote Editing)

If your work is on EC2 and you want to edit it with Cursor:

1. Install Remote-SSH extension in Cursor
2. Connect to EC2 via SSH
3. Open remote folder in Cursor

Note: Cursor's Remote-SSH support behaves like VS Code's Remote-SSH. Availability depends on your Cursor build/version. If Remote-SSH isn't available, use Option 1 (clone from GitHub) or Option 2 (work locally).

For now: Use Option 1 or 2 to work locally.

How to Use Cursor for Your Tasks

Creating Scripts (Instead of vim)

Terminal way:

```
vim scripts/backup-logs.sh
# Type script, save with :wq
```

Cursor way:

1. Open `~/devops-tasks` in Cursor (File → Open Folder)
2. Right-click `scripts/` folder → New File → `backup-logs.sh`
3. Type script (syntax highlighting helps)
4. Save: `Ctrl + S`
5. Done - no vim commands needed

Benefits: Colors help spot errors, auto-indentation, easier to read

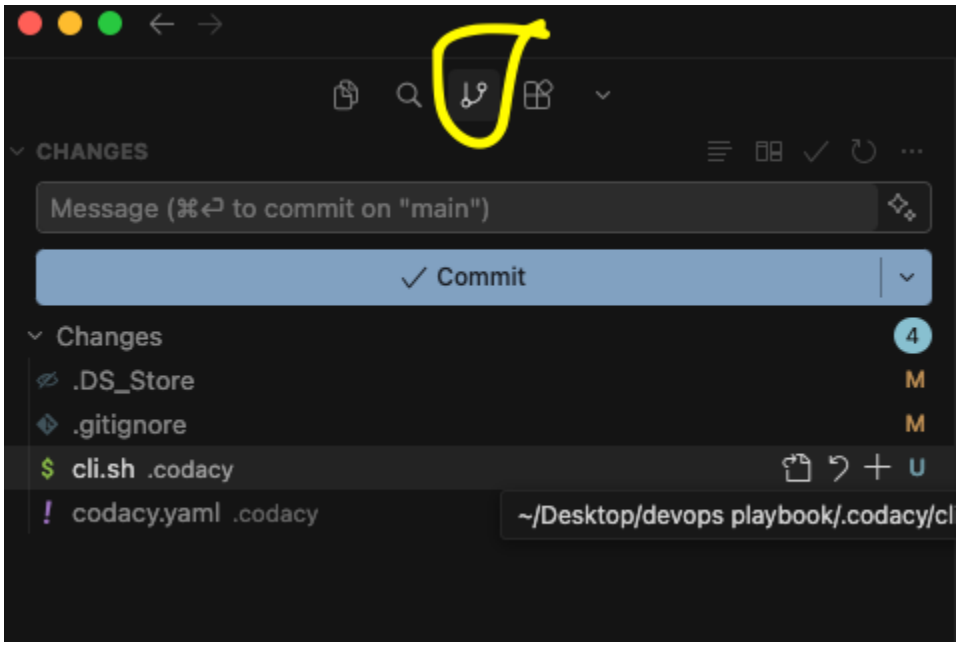
Git Operations (Instead of Terminal Commands)

Terminal way:

```
git add .
git commit -m "Initial commit"
git push origin main
```

Cursor way:

1. Click Source Control icon (left sidebar - looks like branch)



2. See changed files (green = new, yellow = modified)
3. Click + to stage files
4. Type commit message at top
5. Click checkmark (✓) to commit
6. Click "..." → Push to push to GitHub

Benefits: Visual diffs, see changes before committing, less typing

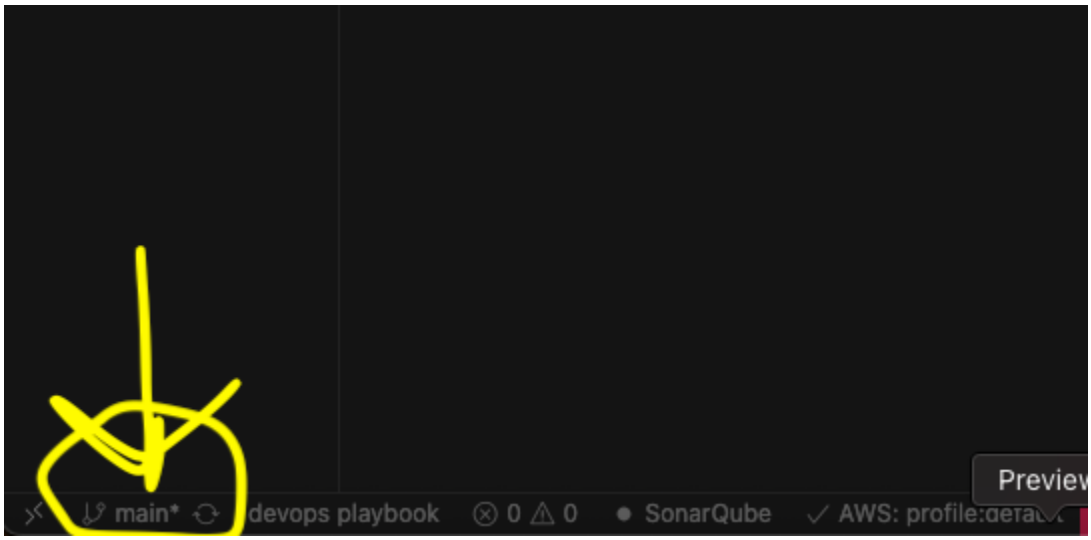
Creating Branches (Instead of git checkout)

Terminal way:

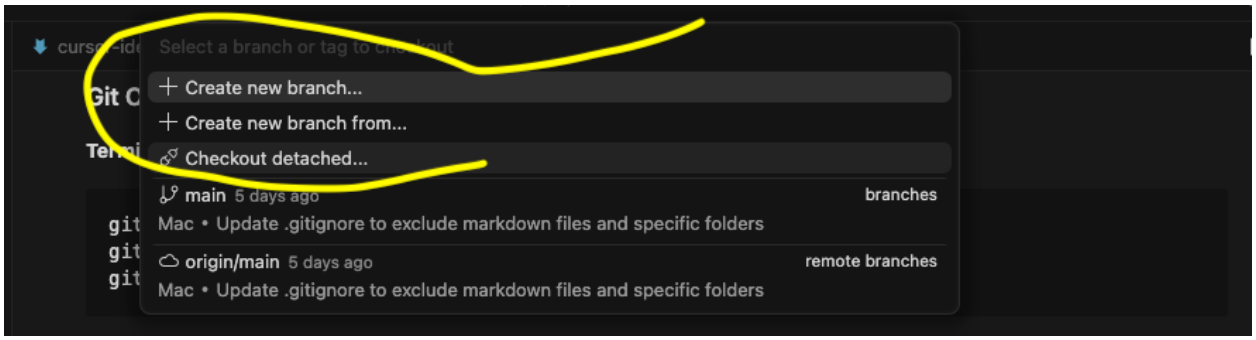
```
git checkout -b feature-name
```

Cursor way:

1. Click branch name (bottom left, shows "main")



2. Click "Create new branch."



3. Type branch name
4. Press Enter

To switch: Click branch name → Select from list ot type git branch (shows list of branches and do git checkout (branch name)

Finding Files and Text (Instead of find/grep)

Terminal way:

```
find. -name "*.sh".
grep -r "8080".
```

Cursor way:

1. Press `Ctrl + P` (or `Cmd + P` on Mac) → Type filename to open
2. Press `Ctrl + Shift + F` → Type text to search across files
3. See all matches listed
4. Click the result to jump to that file/line

Benefits: Visual results, faster than terminal commands

Working with Multiple Files

Terminal way:

```
vim file1.sh
# Edit, save, exit
vim file2.sh
# Edit, save, exit
```

Cursor way:

1. Click files in explorer to open (each opens in a tab)
2. Switch between tabs
3. Edit multiple files at once
4. See all changes in Git panel

Benefits: No need to close/reopen files, see everything at once

Quick Workflow Example

Task: Create a script and commit it

1. **Open project:** File → Open Folder → `~/devops-tasks`
2. **Create script:** Right-click `scripts/` → New File → `test.sh`
3. **Write script:** Type code (syntax highlighting helps)
4. **Save:** `Ctrl + S`
5. **Make executable:** Open terminal in Cursor (`Ctrl + ``) → `chmod +x scripts/test.sh`
6. **Test:** In terminal → `./scripts/test.sh`
7. **Commit:** Click Source Control → Click `+` → Type message → Click ✓
8. **Push:** Click "..." → Push

Result: Created, tested, committed - mostly in Cursor!

Essential Shortcuts

- `Ctrl + `` = Open terminal in Cursor
- `Ctrl + P` = Quick file open (type filename)
- `Ctrl + Shift + F` = Search across files
- `Ctrl + S` = Save file
- `Ctrl + Shift + G` = Open Git panel

When to Use Cursor vs Terminal

Use Cursor for:

- Creating/editing scripts (B1-B4 tasks)
- Git operations (G1-G5 tasks)
- Reading/exploring code
- Working with multiple files
- Search and replace

Use Terminal for:

- SSH to EC2
- Running scripts (`./script.sh`)

- System commands (`systemctl`, `ufw`)
- Quick one-off commands

Use Both:

- Edit in Cursor, test in terminal (same window)
- Best of both worlds

For Future Sections

Section 3 (Docker): Write Dockerfiles in Cursor, build/run in terminal
Section 4 (Kubernetes): Edit YAML files in Cursor, apply with kubectl in the terminal
Section 5 (CI/CD): Write pipeline files in Cursor, trigger in terminal/GitHub
All sections: Use Cursor for code, the terminal for commands

Self Check

- ☐ Can open project folder in Cursor
- ☐ Can create new files (right-click → New File)
- ☐ Can use Git panel to commit changes
- ☐ Can open terminal in Cursor (`Ctrl + ``)
- ☐ Understand when to use Cursor vs terminal

Remember: You can always use the terminal if you prefer. Both work. Use what helps you work faster.